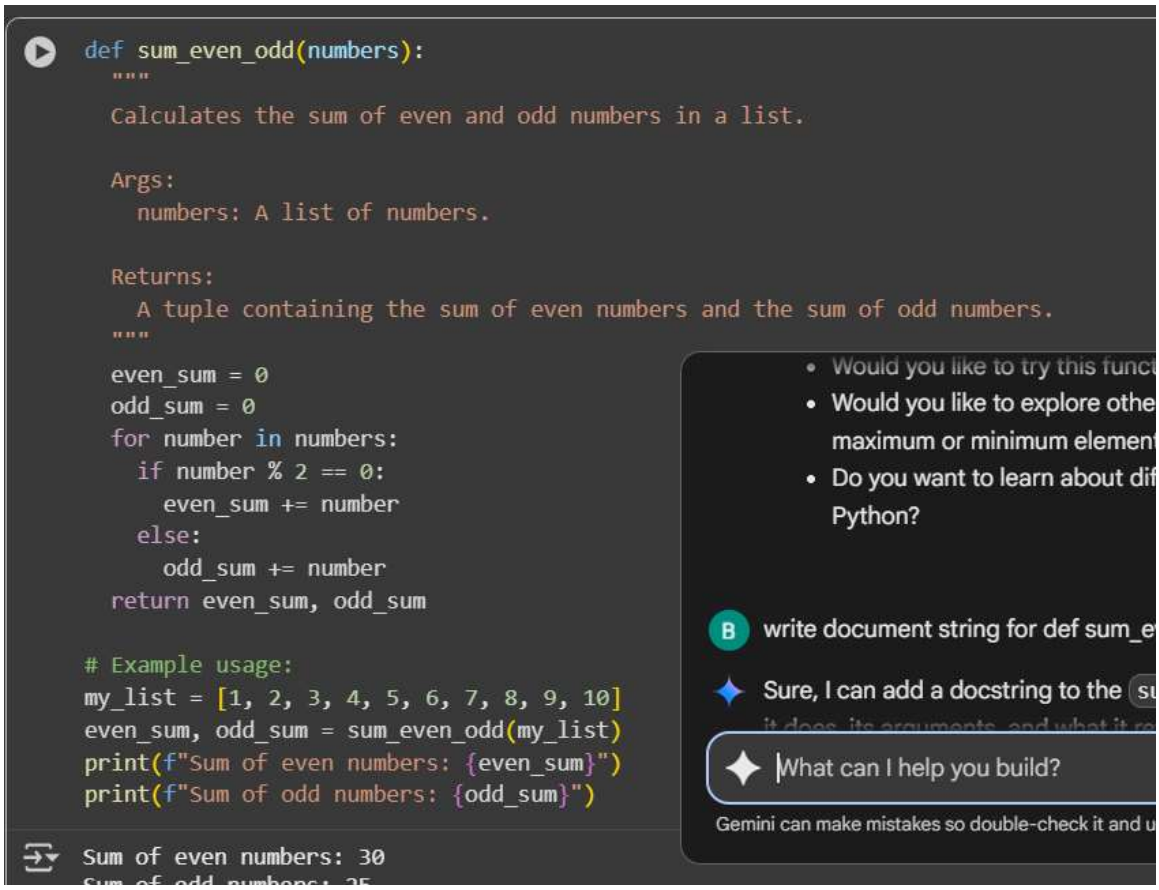
ASSIGNMENT:-9.3

NAME:-PRATHAM KOUSHIK

ROLL NO:-2403A52182

Task Description#1 Basic Docstring Generation
• Write python function to return sum of even and odd numbers in the given list.
• Incorporate manual docstring in code with Google Style
• Use an AI-assisted tool (e.g., Copilot, Cursor AI) to generate a docstring describing
the function.
• Compare the AI-generated docstring with your manually written one

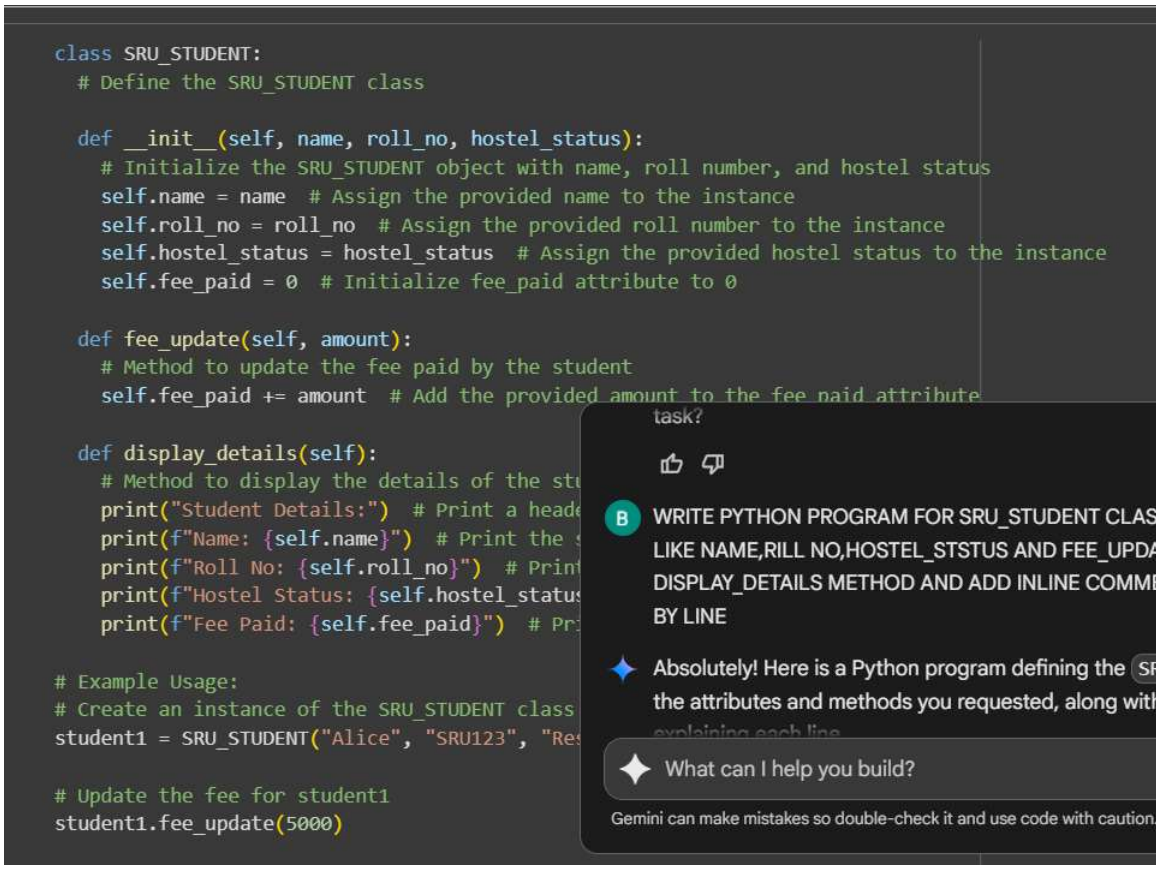
Expected Outcome#1: Students understand how AI can produce function-level documentation


def sum_even_odd(numbers):
"""
Calculates the sum of even and odd numbers in a list.
Args:
numbers: A list of numbers.
Returns:
A tuple containing the sum of even numbers and the sum of odd numbers.
"""
even_sum = 0
odd_sum = 0
for number in numbers:
if number % 2 == 0:
even_sum += number
else:
odd_sum += number
return even_sum, odd_sum
# Example usage:
my_list = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
even_sum, odd_sum = sum_even_odd(my_list)
print(f"Sum of even numbers: {even_sum}")
print(f"Sum of odd numbers: {odd_sum}")
Sum of even numbers: 30
Would you like to try this funct
Would you like to explore othe
maximum or minimum element
Do you want to learn about dif
Python?
B
write document string for def sum_e
Sure, I can add a docstring to the su
What can I help you build?
Gemini can make mistakes so double-check it and u

```python
def sum_even_odd(numbers):
  """
  Calculates the sum of even and odd numbers in a list.

  Args:
    numbers: A list of numbers.

  Returns:
    A tuple containing the sum of even numbers and the sum of odd numbers.
  """
  even_sum = 0
  odd_sum = 0
  for number in numbers:
    if number % 2 == 0:
      even_sum += number
    else:
      odd_sum += number
  return even_sum, odd_sum

# Example usage:
my_list = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
even_sum, odd_sum = sum_even_odd(my_list)
print(f"Sum of even numbers: {even_sum}")
print(f"Sum of odd numbers: {odd_sum}")
```

```
Sum of even numbers: 30
Sum of odd numbers: 25
```

Task Description#2 Automatic Inline Comments
• Write python program for sru_student class with attributes like name, roll no., hostel_status and fee_update method and display_details method.
• Write comments manually for each line/code block
• Ask an AI tool to add inline comments explaining each line/step.
• Compare the AI-generated comments with your manually written one.
Expected Output#2: Students critically analyze AI-generated code comments.



```python
class SRU_STUDENT:
  # Define the SRU_STUDENT class

  def __init__(self, name, roll_no, hostel_status):
    # Initialize the SRU_STUDENT object with name, roll number, and hostel status
    self.name = name  # Assign the provided name to the instance
    self.roll_no = roll_no  # Assign the provided roll number to the instance
    self.hostel_status = hostel_status  # Assign the provided hostel status to the instance
    self.fee_paid = 0  # Initialize fee_paid attribute to 0

  def fee_update(self, amount):
    # Method to update the fee paid by the student
    self.fee_paid += amount  # Add the provided amount to the fee_paid attribute

  def display_details(self):
    # Method to display the details of the stu
    print("Student Details:")  # Print a heade
    print(f"Name: {self.name}")  # Print the
    print(f"Roll No: {self.roll_no}")  # Print
    print(f"Hostel Status: {self.hostel_statu
    print(f"Fee Paid: {self.fee_paid}")  # Pri

# Example Usage:
# Create an instance of the SRU_STUDENT class
student1 = SRU_STUDENT("Alice", "SRU123", "Re

# Update the fee for student1
student1.fee_update(5000)
```

task?

B WRITE PYTHON PROGRAM FOR SRU_STUDENT CLAS
LIKE NAME,RILL NO,HOSTEL_STSTUS AND FEE_UPDA
DISPLAY_DETAILS METHOD AND ADD INLINE COMM
BY LINE

Absolutely! Here is a Python program defining the SR
the attributes and methods you requested, along wit
explaining each line.

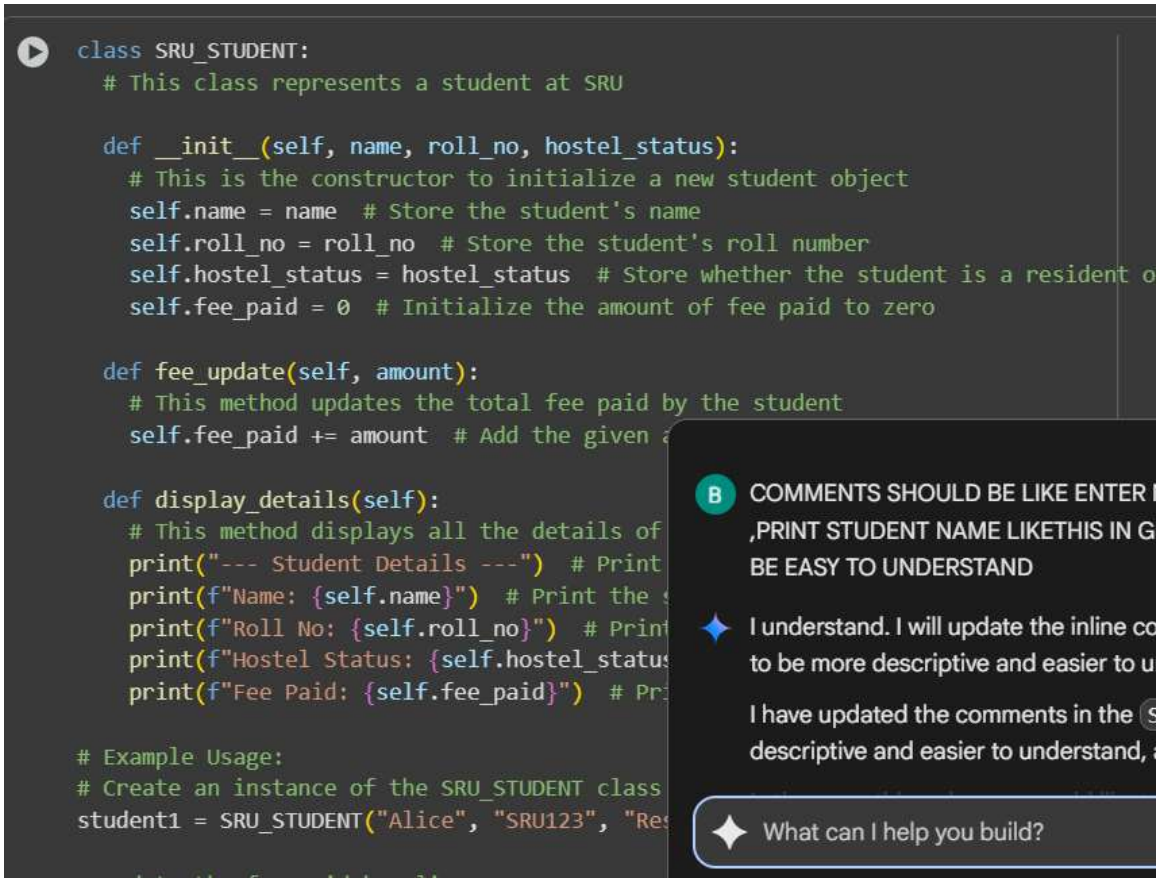
What can I help you build?

Gemini can make mistakes so double-check it and use code with caution.

```python
class SRU_STUDENT:
  # Define the SRU_STUDENT class

  def __init__(self, name, roll_no, hostel_status):
    # Initialize the SRU_STUDENT object with name, roll number, and hostel status
    self.name = name  # Assign the provided name to the instance
    self.roll_no = roll_no  # Assign the provided roll number to the instance
    self.hostel_status = hostel_status  # Assign the provided hostel status to the instance
    self.fee_paid = 0  # Initialize fee_paid attribute to 0

  def fee_update(self, amount):
    # Method to update the fee paid by the student
    self.fee_paid += amount  # Add the provided amount to the fee_paid attribute

  def display_details(self):
    # Method to display the details of the student
    print("Student Details:")  # Print a header for the details
    print(f"Name: {self.name}")  # Print the student's name
    print(f"Roll No: {self.roll_no}")  # Print the student's roll number
    print(f"Hostel Status: {self.hostel_status}")  # Print the student's hostel status
    print(f"Fee Paid: {self.fee_paid}")  # Print the total fee paid by the student

# Example Usage:
# Create an instance of the SRU_STUDENT class
student1 = SRU_STUDENT("Alice", "SRU123", "Resident")

# Update the fee for student1
student1.fee_update(5000)
```

```
[3]
✓ 0s
# Update the fee for student1
student1.fee_update(5000)

# Display the details of student1
student1.display_details()

# Create another instance
student2 = SRU_STUDENT("Bob", "SRU456", "Day Scholar")

# Update fee for student2
student2.fee_update(2000)

# Display details for student2
student2.display_details()
```

```
Student Details:
Name: Alice
Roll No: SRU123
Hostel Status: Resident
Fee Paid: 5000
Student Details:
Name: Bob
Roll No: SRU456
Hostel Status: Day Scholar
Fee Paid: 2000
```



```
class SRU_STUDENT:
  # This class represents a student at SRU

  def __init__(self, name, roll_no, hostel_status):
    # This is the constructor to initialize a new student object
    self.name = name  # Store the student's name
    self.roll_no = roll_no  # Store the student's roll number
    self.hostel_status = hostel_status  # Store whether the student is a resident o
    self.fee_paid = 0  # Initialize the amount of fee paid to zero

  def fee_update(self, amount):
    # This method updates the total fee paid by the student
    self.fee_paid += amount  # Add the given a

  def display_details(self):
    # This method displays all the details of
    print("--- Student Details ---")  # Print
    print(f"Name: {self.name}")  # Print the
    print(f"Roll No: {self.roll_no}")  # Print
    print(f"Hostel Status: {self.hostel_statu
    print(f"Fee Paid: {self.fee_paid}")  # Pri

# Example Usage:
# Create an instance of the SRU_STUDENT class
student1 = SRU_STUDENT("Alice", "SRU123", "Re
```

B COMMENTS SHOULD BE LIKE ENTER
,PRINT STUDENT NAME LIKETHIS IN G
BE EASY TO UNDERSTAND

I understand. I will update the inline co
to be more descriptive and easier to u

I have updated the comments in the
descriptive and easier to understand,

What can I help you build?

```
# Example Usage:
# Create an instance of the SRU_STUDENT class for Alice
student1 = SRU_STUDENT("Alice", "SRU123", "Resident")

# Update the fee paid by Alice
student1.fee_update(5000)

# Display the details of Alice
student1.display_details()

# Create another instance for Bob
student2 = SRU_STUDENT("Bob", "SRU456", "Day Scholar")

# Update fee paid by Bob
student2.fee_update(2000)

# Display details for Bob
student2.display_details()
```

```
--- Student Details ---
Name: Alice
Roll No: SRU123
Hostel Status: Resident
Fee Paid: 5000
--- Student Details ---
Name: Bob
Roll No: SRU456
Hostel Status: Day Scholar
Fee Paid: 2000
```

Task Description#3

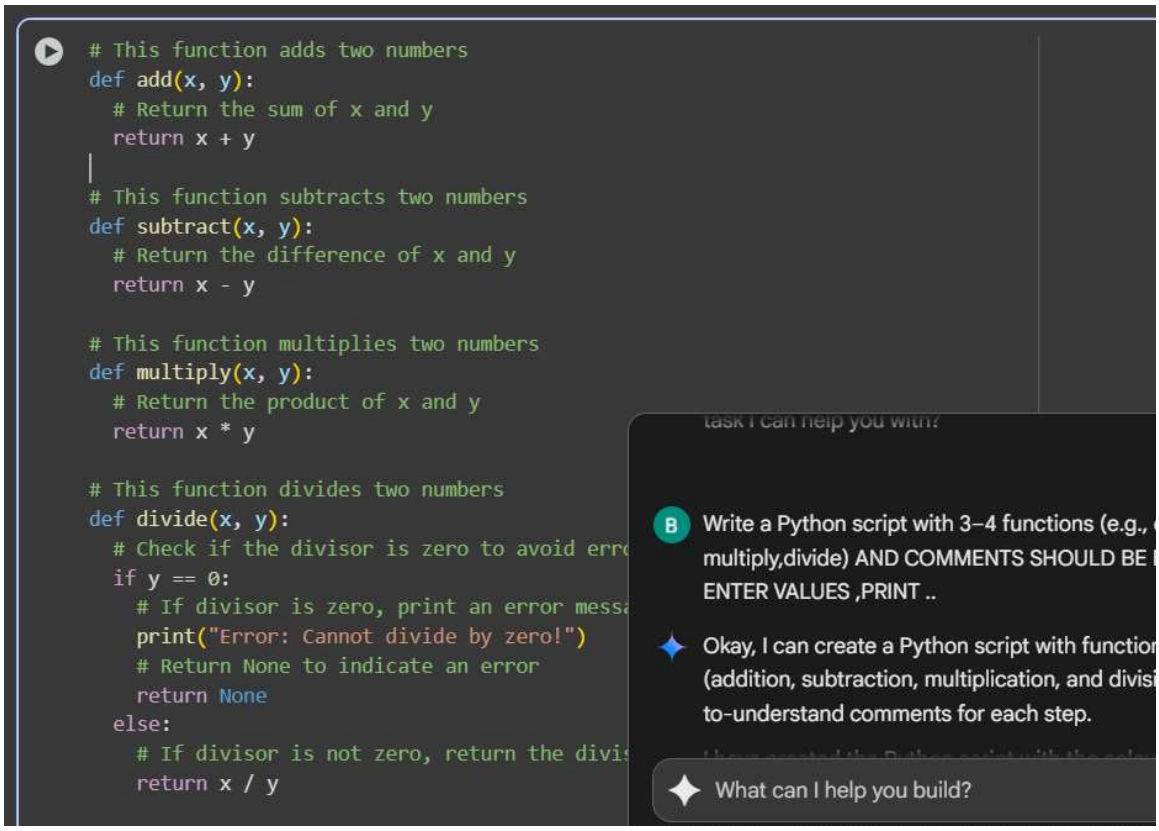
• Write a Python script with 3–4 functions (e.g., calculator: add, subtract, multiply, divide).

• Incorporate manual docstring in code with NumPy Style

• Use AI assistance to generate a module-level docstring + individual function docstrings.

• Compare the AI-generated docstring with your manually written one.

Expected Output#3: Students learn structured documentation for multi-function scripts




```python
# This function adds two numbers
def add(x, y):
  # Return the sum of x and y
  return x + y

# This function subtracts two numbers
def subtract(x, y):
  # Return the difference of x and y
  return x - y

# This function multiplies two numbers
def multiply(x, y):
  # Return the product of x and y
  return x * y

# This function divides two numbers
def divide(x, y):
  # Check if the divisor is zero to avoid erro
  if y == 0:
    # If divisor is zero, print an error messa
    print("Error: Cannot divide by zero!")
    # Return None to indicate an error
    return None
  else:
    # If divisor is not zero, return the divis
    return x / y
```

```python
# This function subtracts two numbers
def subtract(x, y):
  # Return the difference of x and y
  return x - y

# This function multiplies two numbers
def multiply(x, y):
  # Return the product of x and y
  return x * y

# This function divides two numbers
def divide(x, y):
  # Check if the divisor is zero to avoid error
  if y == 0:
    # If divisor is zero, print an error message
    print("Error: Cannot divide by zero!")
    # Return None to indicate an error
    return None
  else:
    # If divisor is not zero, return the division result
    return x / y

# Example Usage:

# Get the first number from the user
num1 = float(input("Enter the first number: "))
# Get the second number from the user
num2 = float(input("Enter the second number: "))
```

```python
# Example Usage:

# Get the first number from the user
num1 = float(input("Enter the first number: "))
# Get the second number from the user
num2 = float(input("Enter the second number: "))

# Perform addition and print the result
print(f"{num1} + {num2} = {add(num1, num2)}")

# Perform subtraction and print the result
print(f"{num1} - {num2} = {subtract(num1, num2)}")

# Perform multiplication and print the result
print(f"{num1} * {num2} = {multiply(num1, num2)}")

# Perform division and print the result
print(f"{num1} / {num2} = {divide(num1, num2)}")

# Example of division by zero (you can test this by entering 0 as the second number)
# num3 = 10
# num4 = 0
# print(f"{num3} / {num4} = {divide(num3, num4)}")
```

```
Enter the first number: 2
Enter the second number: 7
2.0 + 7.0 = 9.0
2.0 - 7.0 = -5.0
2.0 * 7.0 = 14.0
2.0 / 7.0 = 0.2857142857142857
```